SIMATS ENGINEERING

CO2 ASSESSMENT TOOL 2

DEBUGGING AND OPTIMISATION TASK

DATABASE MANAGEMENT SYSTEMS (CSA05)

Assessment detail	Information
Course Name	Database Management Systems
Course Code	CSA05
Course Outcome	CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.
Assessment Tool	Debugging and Optimisation Task
Weightage	10%
Student Name	Pugazharasan K
Registration Number	192511341

Purpose and rubric alignment

This document answers all ten questions and all thirty A/B/C subparts in the uploaded assessment. Each answer identifies the fault, explains its cause, gives corrected SQL or a normalized design, and includes expected results or decomposed records wherever the supplied data supports them.

Rubric criterion	How the answer addresses it
Problem Identification	Names the syntax, reference, grouping, join, subquery, integrity or normalization issue.
Debugging	Provides corrected statements and explains why they work.
Optimization	Removes redundant predicates and discusses correct query structure.

Analysis	Shows calculations, functional dependencies and reasoning.
Code Quality	Uses consistent formatting, aliases, constraints and comments where useful.

Formatting applied: all document text is black; body and SQL text are 14 point; headings are 16 point.

QUESTION 1 – CORRECT CREATE TABLE SYNTAX AND MISSING CONSTRAINTS

Objective: Correct syntax, primary keys, NOT NULL, UNIQUE, CHECK and foreign-key references.

1.A Students table

Error and reasoning: PRIMARY must be PRIMARY KEY. A comma is missing between Name and Age. StudentID must be the primary key, Name mandatory and Email unique.

Corrected SQL:

```
CREATE TABLE Students (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  Age INT,
  Email VARCHAR(80) UNIQUE
);
```

Expected output:

StudentID	Name	Age	Email
101	Asha	19	asha@uni.edu
102	Bala	20	bala@uni.edu

Conclusion: PRIMARY KEY enforces unique, non-null identifiers; NOT NULL requires a name; UNIQUE prevents duplicate email values.

1.B Courses table

Error and reasoning: CHECK needs parentheses and CourseID should be the primary key. Credits must be positive.

Corrected SQL:

```
CREATE TABLE Courses (
  CourseID VARCHAR(5) PRIMARY KEY,
  CourseName VARCHAR(60) NOT NULL,
  Credits INT CHECK (Credits > 0)
);
```

Expected output:

CourseID	CourseName	Credits
C01	DBMS	4
C02	Networks	3

Conclusion: The supplied rows satisfy the constraints. Zero or negative credits are rejected.

1.C Enrollments table

Error and reasoning: Commas and parentheses are missing around the foreign-key declaration, and the Courses foreign key is absent.

Corrected SQL:

```
CREATE TABLE Enrollments (
  EnrollID INT PRIMARY KEY,
  StudentID INT,
  CourseID VARCHAR(5),
  FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
  FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

Expected output:

EnrollID	StudentID	CourseID
1	101	C01
2	102	C02

Conclusion: Students and Courses must exist before this table is created; foreign keys prevent references to nonexistent parent records.

QUESTION 2 – DEBUG AGGREGATES AND GROUP BY

Objective: Use grouping to calculate per-region sales, per-department average salary and per-customer order summaries.

2.A Total sales per region

Error and reasoning: Region is selected with SUM(Amount), so the query needs GROUP BY Region.

Corrected SQL:

```
SELECT Region, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY Region
ORDER BY Region;
```

Expected output:

Region	TotalSales
North	800
South	700

Conclusion: North = 500 + 300 = 800; South = 700.

2.B Departments with average salary above 45000

Error and reasoning: The query uses HAVING but lacks GROUP BY Dept. HAVING filters groups after aggregation.

Corrected SQL:

```
SELECT Dept, AVG(Salary) AS AvgPay
FROM Employees
GROUP BY Dept
HAVING AVG(Salary) > 45000
ORDER BY Dept;
```

Expected output:

Dept	AvgPay
IT	50000.00

Conclusion: IT average = (45,000 + 55,000)/2 = 50,000. HR average = 40,000 and is excluded.

2.C Order count and total per customer

Error and reasoning: ORDER BY sorts rows but does not aggregate them. GROUP BY CustomerID creates one group per customer.

Corrected SQL:

```
SELECT CustomerID, COUNT(*) AS OrdersCount,
       SUM(Total) AS OrderTotal
FROM Orders
GROUP BY CustomerID
ORDER BY CustomerID;
```

Expected output:

CustomerID	OrdersCount	OrderTotal
201	2	250
202	1	300

Conclusion: Customer 201 has two orders totalling 250; customer 202 has one order totalling 300.

QUESTION 3 – CORRECT MISSPELLED OR IMPROPERLY REFERENCED COLUMNS

Objective: Replace column names that do not exist in the supplied relation definitions.

3.A Correct StudntID

Error and reasoning: StudntID is misspelled; the valid column is StudentID.

Corrected SQL:

```
SELECT StudentID, StudentName
FROM Student;
```

Expected output:

StudentID	StudentName
1	Divya
2	Farhan

Conclusion: Column names must match the defined schema.

3.B Replace Cost

Error and reasoning: Product contains Price, not Cost.

Corrected SQL:

```
SELECT ProductName, Price
FROM Product;
```

Expected output:

ProductName	Price
Pen	20
Book	80

Conclusion: Selecting Price avoids an unknown-column error.

3.C Correct Faculty column

Error and reasoning: The schema defines FacultyName, not Faculty_Name.

Corrected SQL:

```
SELECT Faculty.FacultyName, Faculty.Department
FROM Faculty;
```

Expected output:

FacultyName	Department
Meena	CSE
Ravi	ECE

Conclusion: Qualification with the table name is valid when the column itself exists.

QUESTION 4 – REPAIR INCORRECT OR MISSING JOIN CONDITIONS

Objective: Join relations using attributes that represent the same key relationship.

4.A Match students to scores

Error and reasoning: The faulty query compares StudentID with Score. The shared relationship is StudentID in both tables.

Corrected SQL:

```
SELECT s.Name, m.Score
FROM Students AS s
INNER JOIN Marks AS m
  ON s.StudentID = m.StudentID
ORDER BY s.StudentID;
```

Expected output:

Name	Score
Asha	85
Bala	91

Conclusion: Matching key columns associates each student with the correct score.

4.B Add the Customers–Orders condition

Error and reasoning: A JOIN without ON may produce a Cartesian product. Match CustomerID between Customers and Orders.

Corrected SQL:

```
SELECT c.Name, o.OrderID
FROM Customers AS c
INNER JOIN Orders AS o
  ON c.CustomerID = o.CustomerID
ORDER BY o.OrderID;
```

Expected output:

Name	OrderID
Ana	101

Ben	102

Conclusion: The join returns only each customer's related orders.

4.C Match staff to departments

Error and reasoning: StaffID and DeptID identify different entities. Staff.DeptID must match Departments.DeptID.

Corrected SQL:

```
SELECT s.StaffName, d.DeptName
FROM Staff AS s
INNER JOIN Departments AS d
  ON s.DeptID = d.DeptID
ORDER BY s.StaffID;
```

Expected output:

StaffName	DeptName
Leela	CSE
Kiran	EEE

Conclusion: Leela belongs to department 1 (CSE); Kiran belongs to department 2 (EEE).

QUESTION 5 – FIX SCALAR SUBQUERIES RETURNING MULTIPLE ROWS

Objective: Use multi-value operators when a subquery may return more than one value.

5.A Departments returned by a subquery

Error and reasoning: The subquery returns DeptID 10 and 20, so equality to a scalar result is incorrect. IN supports multiple values.

Corrected SQL:

```
SELECT EmpID
FROM Employees
WHERE DeptID IN (SELECT DeptID FROM Departments)
ORDER BY EmpID;
```

Expected output:

EmpID
1
2
3

Conclusion: All three employees belong to one of the listed departments.

5.B Multiple product prices

Error and reasoning: The inner query may return two rows priced 50. IN correctly handles a set of prices.

Corrected SQL:

```
SELECT ProductID
FROM Products
WHERE Price IN (
  SELECT Price FROM Products WHERE Price <= 50
)
ORDER BY ProductID;
```

Expected output:

ProductID
1

3

Conclusion: Products 1 and 3 cost 50. A simpler direct query for the stated condition is WHERE Price <= 50.

5.C Eligible student departments

Error and reasoning: EligibleDept contains CSE and IT, so IN is required instead of a scalar equality.

Corrected SQL:

```
SELECT StudentID
FROM Students
WHERE Dept IN (SELECT Dept FROM EligibleDept)
ORDER BY StudentID;
```

Expected output:

StudentID
1
2

Conclusion: Students 1 and 2 are in eligible departments; student 3 is in ECE.

QUESTION 6 – CORRECT CREATE VIEW REFERENCES OR SYNTAX

Objective: Use valid column names and the required CREATE VIEW ... AS SELECT syntax.

6.A HighPay view

Error and reasoning: Employees has Salary, not Pay. The filter can use Salary even though Salary is not displayed.

Corrected SQL:

```
CREATE VIEW HighPay AS
SELECT EmpName
FROM Employees
WHERE Salary > 50000;

SELECT * FROM HighPay;
```

Expected output:

EmpName
Bob

Conclusion: Bob earns 52,000 and qualifies; Amy earns 48,000 and does not.

6.B CreditCourses view

Error and reasoning: The keyword AS is missing between the view name and SELECT.

Corrected SQL:

```
CREATE VIEW CreditCourses AS
SELECT Title, Credits
FROM Courses
WHERE Credits >= 4;

SELECT * FROM CreditCourses;
```

Expected output:

Title	Credits
DBMS	4

Conclusion: DBMS has four credits; OS has three and is excluded.

6.C TopStudents view

Error and reasoning: The Students schema contains Name, not StudentName.

Corrected SQL:

```
CREATE VIEW TopStudents AS
SELECT Name, CGPA
FROM Students
WHERE CGPA >= 8;

SELECT * FROM TopStudents;
```

Expected output:

Name	CGPA
Nila	8.5

Conclusion: Nila meets the threshold; Ram's CGPA is below 8.

QUESTION 7 – SIMPLIFY REDUNDANT WHERE CONDITIONS

Objective: Remove repeated or logically implied predicates without changing results under the assumptions stated in the question.

7.A Inventory filters

Error and reasoning: Qty > 0 implies Qty >= 1 because Qty is an integer. Price >= 0 is guaranteed by the schema and appears twice, so both copies are unnecessary.

Corrected SQL:

```
SELECT *
FROM Inventory
WHERE Qty > 0;
```

Expected output:

ItemID	Qty	Price
1	5	100
3	12	50

Conclusion: Only rows with positive quantity remain.

7.B Student CGPA and department

Error and reasoning: CGPA >= 8 implies CGPA >= 7, and Dept = 'CSE' is repeated. Retain the stronger CGPA test and one department test.

Corrected SQL:

```
SELECT *
FROM Students
WHERE CGPA >= 8 AND Dept = 'CSE';
```

Expected output:

StudentID	CGPA	Dept
1	8.2	CSE
3	9.1	CSE

Conclusion: Students 1 and 3 satisfy both conditions.

7.C Order amount filter

Error and reasoning: Amount > 500 implies Amount > 100. Remove the weaker comparison but retain Status = 'PAID'.

Corrected SQL:

```
SELECT *
FROM Orders
WHERE Amount > 500 AND Status = 'PAID';
```

Expected output:

OrderID	Amount	Status
3	700	PAID

Conclusion: Only order 3 is paid and exceeds 500.

QUESTION 8 – IDENTIFY AND CORRECT INTEGRITY VIOLATIONS

Objective: Identify primary-key, NOT NULL, CHECK, UNIQUE and foreign-key violations.

8.A Student integrity constraints

Error and reasoning: The existing StudentID 1 for Asha means inserting another ID 1 violates the primary key. (3,NULL,16) violates Name NOT NULL and Age CHECK(Age>=17).

Corrected SQL:

```
-- Assuming the existing row (1, 'Asha', 19) is retained:
INSERT INTO Students (StudentID, Name, Age)
VALUES (2, 'Bala', 20);

INSERT INTO Students (StudentID, Name, Age)
VALUES (3, 'Charu', 18);
```

Expected output:

StudentID	Name	Age
1	Asha	19
2	Bala	20
3	Charu	18

Conclusion: Charu and age 18 are proposed valid replacements because the source does not state the intended correct values.

8.B SKU uniqueness and positive price

Error and reasoning: SKU P01 duplicates product 1, violating UNIQUE. Price -5 violates CHECK(Price>0).

Corrected SQL:

```
-- Assuming product 1 with SKU P01 remains:
INSERT INTO Products (ProductID, SKU, Price)
VALUES (2, 'P02', 25.00);
```

Expected output:

ProductID	SKU	Price

1	P01	50
2	P02	25.00

Conclusion: P02 and 25.00 are proposed valid replacements, not facts from the source.

8.C Foreign-key department violation

Error and reasoning: Employee DeptID 99 references no Department row; only 10 and 20 exist.

Corrected SQL:

```
-- If employee 2 belongs to department 20:
INSERT INTO Employee (EmpID, DeptID)
VALUES (2, 20);
```

Expected output:

EmpID	DeptID
1	10
2	20

Conclusion: 20 is an illustrative correction because it exists in Department. Confirm the employee's actual department before choosing 10 or 20.

QUESTION 9 – NORMALIZE 1NF AND 2NF VIOLATIONS

Objective: Remove repeating groups and partial dependencies from the supplied relations.

9.A StudentContacts to 1NF

Error and reasoning: PhoneNumbers stores multiple comma-separated values in one attribute. Store one atomic phone number per row.

Corrected SQL:

```
CREATE TABLE StudentContactNumbers (
  StudentID INT NOT NULL,
  Name VARCHAR(80) NOT NULL,
  PhoneNumber VARCHAR(20) NOT NULL,
  PRIMARY KEY (StudentID, PhoneNumber)
);
```

Expected output:

StudentID	Name	PhoneNumber
1	Asha	9001
1	Asha	9002
2	Bala	9003

Conclusion: Each phone number is atomic. A cleaner design can store StudentID and Name once in Student, and only StudentID plus PhoneNumber in StudentContactNumbers.

9.B Enrollment to 2NF

Error and reasoning: Composite key is (StudentID, CourseID). StudentID→StudentName and CourseID→CourseTitle are partial dependencies; Grade depends on the full key.

Corrected SQL:

```
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(80) NOT NULL
);
CREATE TABLE Course (
  CourseID VARCHAR(10) PRIMARY KEY,
```

```
  CourseTitle VARCHAR(100) NOT NULL
);
CREATE TABLE Enrollment (
  StudentID INT NOT NULL,
  CourseID VARCHAR(10) NOT NULL,
  Grade VARCHAR(5),
  PRIMARY KEY (StudentID, CourseID),
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
  FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

Expected output:

Relation	Sample records
Student	(1, Asha); (2, Bala)
Course	(C1, DBMS); (C2, OS)
Enrollment	(1,C1,A); (1,C2,B); (2,C1,A)

Conclusion: StudentName and CourseTitle are stored once in their respective tables; Grade stays in Enrollment because it depends on the full composite key.

9.C Normalize OrderLine

Error and reasoning: OrderID→OrderDate and ProductID→ProductName are partial dependencies on the composite key (OrderID, ProductID). Qty depends on the full key.

Corrected SQL:

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  OrderDate DATE NOT NULL
);
CREATE TABLE Product (
  ProductID VARCHAR(10) PRIMARY KEY,
  ProductName VARCHAR(100) NOT NULL
);
CREATE TABLE OrderLine (
  OrderID INT NOT NULL,
  ProductID VARCHAR(10) NOT NULL,
```

```
    Qty INT NOT NULL CHECK (Qty > 0),
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
  );
```

Expected output:

Relation	Sample records
Orders	(10, 2026-08-01)
Product	(P1, Pen); (P2, Book)
OrderLine	(10,P1,2); (10,P2,1)

Conclusion: OrderDate is stored in Orders, ProductName in Product, and Qty in OrderLine. This removes partial dependencies.

QUESTION 10 – REMOVE REDUNDANCY THROUGH 3NF

Objective: Remove transitive dependencies and identify primary and foreign keys.

10.A Employee and department normalization

Error and reasoning: EmpID→EmpName,DeptID and DeptID→DeptName mean DeptName depends transitively on EmpID. Store department details separately.

Corrected SQL:

```
CREATE TABLE Department (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(80) NOT NULL
);
CREATE TABLE Employee (
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(80) NOT NULL,
  DeptID INT NOT NULL,
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Expected output:

Relation	Sample records
Department	(10,CSE); (20,IT)
Employee	(1,Asha,10); (2,Bala,10); (3,Cara,20)

Conclusion: DeptID is the Department primary key and Employee foreign key. Department names are stored once.

10.B Student and advisor normalization

Error and reasoning: StudentID→Name,AdvisorID and AdvisorID→AdvisorPhone create a transitive dependency. Store advisor phone details separately.

Corrected SQL:

```
CREATE TABLE Advisor (
  AdvisorID INT PRIMARY KEY,
  AdvisorPhone VARCHAR(20) NOT NULL
);
```

```
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(80) NOT NULL,
  AdvisorID INT NOT NULL,
  FOREIGN KEY (AdvisorID) REFERENCES Advisor(AdvisorID)
);
```

Expected output:

Relation	Sample records
Advisor	(7,90001); (8,90002)
Student	(1,Nila,7); (2,Arun,7); (3,Ria,8)

Conclusion: AdvisorPhone is stored once per advisor. AdvisorID is the Advisor primary key and a Student foreign key.

10.C Book and publisher normalization

Error and reasoning: BookID→Title,PublisherID and PublisherID→PublisherCity make PublisherCity transitively dependent on BookID. Store publisher city in Publisher.

Corrected SQL:

```
CREATE TABLE Publisher (
  PublisherID VARCHAR(10) PRIMARY KEY,
  PublisherCity VARCHAR(80) NOT NULL
);
CREATE TABLE Book (
  BookID VARCHAR(10) PRIMARY KEY,
  Title VARCHAR(100) NOT NULL,
  PublisherID VARCHAR(10) NOT NULL,
  FOREIGN KEY (PublisherID) REFERENCES Publisher(PublisherID)
);
```

Expected output:

Relation	Sample records
Publisher	(P1,Chennai); (P2,Delhi)
Book	(B1,DBMS,P1); (B2,SQL,P1); (B3,AI,P2)

Conclusion: PublisherID is the Publisher primary key and a Book foreign key. PublisherCity is stored once per publisher, reducing update anomalies.

FINAL SUMMARY AND SUBMISSION CHECKLIST

The assessment covers SQL syntax correction, grouping and aggregates, valid column references, join conditions, multi-row subqueries, view definitions, predicate simplification, integrity constraints, and normalization through 1NF, 2NF and 3NF.

Important concepts

- PRIMARY KEY uniquely identifies rows; FOREIGN KEY enforces valid references.
- NOT NULL, UNIQUE and CHECK enforce completeness, uniqueness and domain rules.
- GROUP BY creates groups for aggregate calculations; HAVING filters aggregated groups.
- Join related tables on matching keys, not unrelated attributes.
- Use IN when a subquery may return multiple values; use equality only when a single scalar value is guaranteed.
- CREATE VIEW requires AS before SELECT and valid source column names.
- Remove redundant predicates only when logical implication or stated schema constraints guarantee the same results.
- 1NF removes repeating groups; 2NF removes partial dependencies on a composite key; 3NF removes transitive dependencies.

Before submission

- Fill in your name and registration number.
- Question 8 includes proposed replacement values because the assessment does not specify the intended correct data for every invalid row.
- Test statements in MySQL 8.0 if your instructor requires execution evidence.
- Follow institutional academic-integrity requirements.

References

1. SIMATS Engineering, Database Management Systems (CSA05), CO2 Assessment Tool 2 – Debugging and Optimisation Task, uploaded PDF.

2. MySQL 8.0 Reference Manual: CREATE TABLE, constraints, SELECT, aggregate functions, GROUP BY, JOIN, subqueries, CREATE VIEW and data integrity.

3. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems – relational design, functional dependencies and normalization.

4. Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts – relational model and SQL.